Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Escuela de Ingeniería Electrónica
Departamento de Sistemas e Informática

SISTEMAS EMBEBIDOS AVANZADOS II

Guía práctica: build system

Del botón Build al binario: compilación a mano, con Make y con CMake sobre la FRDM-KL46Z

Prof. Ing. Daniel Márquez
dmarquez@fceia.unr.edu.ar
Octubre de 2026 — Versión 1.0

Índice

1. Objetivo de la práctica 4
1.1. ¿Para qué entender el build, si el IDE lo hace solo? 4
2. Conceptos previos 5
2.1. IDE, toolchain, build system y depurador 5
2.2. Build y build system 5
2.3. Las etapas del build 5
2.4. Qué es cada archivo 6
2.5. Make, CMake y Ninja: quién reemplaza lo que hacemos a mano 6
2.6. Debug y Release: niveles de optimización 7
3. El proyecto blinky_kl46z 8
3.1. main.c 8
3.2. El startup: lo que pasa antes de main() 9
3.3. El linker script: el archivo que describe el chip 9
4. Preparación del entorno en Windows 11
4.1. Herramientas necesarias 11
4.2. Descomprimir el proyecto 11
4.3. Abrir la consola en la carpeta del proyecto 11
4.4. Configurar el PATH 11
5. Práctica 1: el build a mano 13
5.1. Qué hace cada comando 13
5.2. Qué significa cada opción 13
5.3. Qué observar en cada archivo 14
5.4. Experimento: linkear sin -nostdlib 15
6. Práctica 2: explorar el binario 16
6.1. Cuánta memoria ocupa 16
6.2. Dónde quedó cada sección 16
6.3. Experimento: la RAM no alcanza 16
7. Práctica 3: automatizar con Make 17
7.1. El Makefile 17
7.2. Paso a paso 18
8. Práctica 4: CMake y Ninja 20
8.1. El toolchain file 20
8.2. El CMakeLists.txt 20
8.3. Paso a paso 21
9. Grabar el programa en la placa 22
10. Del blinky al IDE y a los SDK 22
10.1. El botón Build del MCUXpresso 22
10.2. ESP-IDF y Zephyr 22

11. Problemas frecuentes 24
12. Referencia rápida de comandos 24
13. Ejercicios 25
14. Referencias 25

1. Objetivo de la práctica

Cuando en un IDE como el MCUXpresso se presiona **Build**, en pocos segundos aparece un archivo listo para grabar en el microcontrolador. Esta práctica abre esa caja negra. Con un único programa —un LED que parpadea en la FRDM-KL46Z, escrito sin SDK ni bibliotecas— se recorre el proceso de compilación tres veces:

- **A mano**, ejecutando uno por uno los comandos del compilador (práctica 1).
- **Con Make**, la herramienta que usa el MCUXpresso por dentro (práctica 3).
- **Con CMake y Ninja**, las herramientas que usan ESP-IDF, Zephyr y la mayoría de los SDK actuales (práctica 4).

En el medio se exploran el **startup** (lo que se ejecuta antes de `main()`) y el **linker script** (el archivo que decide dónde queda cada cosa en la memoria del chip), que son las dos piezas propias de los sistemas embebidos.

Esta guía acompaña a la presentación de la clase. Sirve para seguir las demostraciones, para repetirlas en casa y como material de consulta. Todos los comandos y resultados que aparecen fueron verificados.

1.1. ¿Para qué entender el build, si el IDE lo hace solo?

El IDE compila por nosotros. Entender cómo lo hace permite:

Capacidad	Ejemplos concretos
Diagnosticar problemas	Entender errores de link (`undefined reference`, `multiple definition`) y de memoria (`region RAM overflowed`); un *HardFault* por desborde de stack; un depurador que "salta" líneas o muestra variables como *optimized out*.
Controlar la memoria	Saber qué ocupa FLASH y qué ocupa RAM, que en un microcontrolador son escasas. El heap de FreeRTOS (`configTOTAL_HEAP_SIZE`) y los stacks de las tareas salen de la RAM: en el archivo `.map` se ve cuánto consume el RTOS.
Ubicar el código donde se necesita	Dividir la FLASH entre un bootloader y la aplicación (actualizaciones OTA), guardar configuración en una dirección fija, ejecutar funciones desde RAM. Todo eso se hace en el linker script; la tabla de particiones de ESP-IDF es la misma idea.
Entender la optimización	Debug y Release no generan el mismo código. El optimizador elimina y reordena instrucciones; por eso existe `volatile`. En esta práctica aparece un caso real en el startup.
No depender de un IDE ni de un fabricante	MCUXpresso, STM32CubeIDE, ESP-IDF y Zephyr usan por debajo el mismo `gcc`, el mismo linker y Make o CMake. Quien entiende el build se mueve entre plataformas sin empezar de cero y puede leer el build de cualquier SDK.
Trabajar en equipo y en la industria	Builds reproducibles (que el firmware compile igual en cualquier PC), integración continua (compilar automáticamente en un servidor, donde no hay IDE) y trazabilidad (saber con qué compilador y opciones se generó el binario de un producto).
Cuidar el hardware	Saber exactamente qué se graba. En los Kinetis, un valor erróneo en el *flash configuration field* puede **bloquear el chip** de forma permanente.

2. Conceptos previos

2.1. IDE, toolchain, build system y depurador

Una confusión habitual es pensar que "el MCUXpresso compila". En realidad, el IDE **llama** a otros programas, que son los que trabajan. Son cuatro piezas distintas:

Pieza	Qué hace	En la KL46Z	En el ESP32-S3
IDE	Aplicación con editor, botones y ventanas de depuración. **No compila por sí misma:** llama a las demás piezas y muestra los resultados.	MCUXpresso IDE	VS Code + extensión ESP-IDF
Build system	Herramienta que **decide qué compilar, en qué orden y con qué opciones**, y llama a la toolchain.	`make`	`cmake` + `ninja`
Toolchain	Conjunto de programas de línea de comandos que **traducen y linkean**: el compilador propiamente dicho.	`arm-none-eabi-gcc` y compañía	`xtensa-esp32s3-elf-gcc`
Depurador	Graba el chip, detiene el programa, muestra variables.	`gdb` + sonda OpenSDA	`gdb` + USB-JTAG

```
Botón Build (IDE)
      │
      ▼
Build system (make / cmake + ninja) ── decide qué hay que recompilar
      │
      ▼
Toolchain (arm-none-eabi-gcc ...)  ── traduce y linkea
      │
      ▼
blinky.elf ──► depurador ──► placa
```

> **Dato concreto:** el `arm-none-eabi-gcc` y el `make` que se usan en esta práctica desde la línea de comandos son **los mismos** que trae instalados el MCUXpresso (carpetas `ide\tools\bin` e `ide\buildtools\bin`). El IDE no hace nada distinto: solo lo esconde detrás de un botón.

2.2. Build y build system

- **Build** (compilación) es el **proceso**: preprocesar, compilar, ensamblar, linkear y convertir.
- **Build system** es la **herramienta que automatiza** ese proceso: decide qué pasos ejecutar y con qué opciones.

En la práctica 1 se hace el build completo **sin build system**: nosotros cumplimos ese rol, eligiendo el orden, los archivos y las opciones de cada comando. En las prácticas 3 y 4 ese mismo build lo ejecutan Make y CMake.

2.3. Las etapas del build

```
main.c ──► main.i ──► main.s ──► main.o ──► blinky.elf ──► blinky.bin
      (1)       (2)       (3)       (4)            (5)
```

Etapa	Programa (opción de gcc)	Qué hace
1. Preprocesar	preprocesador (`gcc -E`)	Reemplaza texto: cada `#include` por el contenido del archivo y cada `#define` por su valor. El compilador nunca ve las macros.

Etapa	Programa (opción de gcc)	Qué hace
2. Compilar	compilador (`gcc -S`)	Traduce C a assembler del procesador destino.
3. Ensamblar	ensamblador (`gcc -c`)	Traduce el assembler a código máquina. El resultado (`.o`) **todavía no tiene direcciones finales**.
4. Linkear	linker (`gcc -T script.ld`)	Junta todos los `.o`, **asigna las direcciones** según el linker script y resuelve las referencias entre archivos.
5. Convertir	`objcopy -O binary`	Extrae del `.elf` la imagen cruda que se graba en la FLASH.

`gcc` es en realidad un **programa director** (*driver*): según la opción que recibe, ejecuta el preprocesador, el compilador, el ensamblador o el linker. Con la opción `-v` muestra las llamadas reales.

2.4. Qué es cada archivo

Archivo	Qué es	Lo genera	Para qué sirve
`.c`	Código fuente	Nosotros	Lo que escribimos
`.i`	C preprocesado: sin `#include` ni `#define`	Preprocesador	Ver qué recibe realmente el compilador
`.s`	Assembler del Cortex-M0+	Compilador	Ver qué instrucciones se generaron
`.o`	Código máquina **sin direcciones finales** (archivo objeto)	Ensamblador	Pieza intermedia: uno por cada `.c`
`.elf`	Programa completo: código, direcciones, símbolos e información de depuración. En el MCUXpresso tiene extensión `.axf`	Linker	Depurar
`.bin`	Imagen cruda de la FLASH, byte a byte	`objcopy`	Grabar en el chip
`.map`	Informe del linker: dónde quedó cada función y variable, y qué descartó	Linker (opción `-Map`)	Analizar el uso de FLASH y RAM

2.5. Make, CMake y Ninja: quién reemplaza lo que hacemos a mano

Quién	Lee	Qué hace
Nosotros	—	Escribimos cada comando de `gcc` y decidimos el orden (práctica 1).
`make`	`Makefile`	Ejecuta **los mismos comandos de gcc**, pero solo los necesarios: compara la fecha de cada `.c` con la de su `.o` y recompila únicamente lo que cambió.
`cmake`	`CMakeLists.txt`	**No compila.** A partir de una descripción del proyecto elige la toolchain y las opciones y **escribe las instrucciones** para que otro programa compile (`build.ninja` o un `Makefile`).
`ninja`	`build.ninja`	Ejecuta las instrucciones que escribió `cmake`. Hace lo mismo que `make`, pero más rápido y con archivos pensados para ser generados, no escritos a mano.

Una analogía: hacer el build a mano es cocinar siguiendo la receta paso a paso. `make` es un cocinero que sigue una receta escrita (el `Makefile`) y no repite lo que ya está hecho. `cmake` es quien escribe la receta, y `ninja` el cocinero que la sigue.

> **Idea clave: `make` y `ninja` ejecutan; `cmake` genera.** Y todos, al final, llaman a `arm-none-eabi-gcc`: ninguno compila por sí mismo. El botón Build del MCUXpresso usa `make`; VS Code con ESP-IDF usa `cmake` + `ninja`.

2.6. Debug y Release: niveles de optimización

Un mismo programa se puede compilar de distintas formas según las **opciones de optimización** que se le pasan a `gcc`. Debug y Release no son modos especiales del compilador: son nombres para dos combinaciones de opciones.

Opción	Efecto
`-O0`	Sin optimización. El código máquina sigue al código C línea por línea. Ideal para depurar.
`-Og`	Optimiza sin estorbar la depuración.
`-O1`, `-O2`, `-O3`	Optimización creciente, orientada a la velocidad.
`-Os`	Optimiza el **tamaño**. Muy usada en microcontroladores.
`-g`, `-g3`	Incluyen información de depuración en el `.elf` (no cambian el código grabado). `-g3` agrega también las macros.

Modo	Opciones en este proyecto	Características
Release	`-Os -g`	Código chico y rápido; el optimizador reordena y elimina instrucciones. Es lo que se graba en un producto. 380 bytes de código.
Debug	`-O0 -g3`	Código tal cual se escribió: los breakpoints caen donde corresponde y se ven todas las variables. 472 bytes de código.

3. El proyecto blinky_kl46z

Es un programa mínimo que hace parpadear el **LED rojo** de la FRDM-KL46Z (pin PTE29), escrito **sin SDK ni HAL**: los registros del microcontrolador se acceden por su dirección, tomada del manual de referencia del KL46. Así, todo lo que termina en el binario está en estos archivos:

```
blinky_kl46z\
├── src\main.c                    Programa principal
├── startup\startup_mkl46z4.c     Tabla de vectores, flash config y Reset_Handler
├── linker\mkl46z256.ld           Linker script
├── Makefile                      Build con Make
├── CMakeLists.txt                Build con CMake
├── cmake\arm-none-eabi.cmake     Toolchain file de CMake
├── entorno.bat                   Configura el PATH en Windows
└── README.md
```

3.1. main.c

```
/*
 * Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
 * Blinky bare-metal para la FRDM-KL46Z (MKL46Z256VLL4)
 *
 * Sin SDK ni HAL: los registros se acceden por su dirección,
 * tomada del Reference Manual del KL46.
 *
 * Hace parpadear el LED rojo de la placa (PTE29, activo en bajo).
 */

#include <stdint.h>

/* ---------- Registros (KL46 Sub-Family Reference Manual) ---------- */

/* SIM: habilitación de relojes de los puertos */
#define SIM_SCGC5        (*(volatile uint32_t *) 0x40048038u)
#define SIM_SCGC5_PORTE  (1u << 13)

/* PORTE: Pin Control Register del pin 29 */
#define PORTE_PCR29      (*(volatile uint32_t *) (0x4004D000u + 4u * 29u))
#define PORT_PCR_MUX_GPIO (1u << 8)            /* MUX = 001: el pin funciona como GPIO */

/* GPIOE */
#define GPIOE_PSOR       (*(volatile uint32_t *) 0x400FF104u)   /* pone pines en 1      */
#define GPIOE_PTOR       (*(volatile uint32_t *) 0x400FF10Cu)   /* invierte pines       */
#define GPIOE_PDDR       (*(volatile uint32_t *) 0x400FF114u)   /* dirección: 1 = salida */

#define LED_ROJO         (1u << 29)

/* ---------- Variables globales ---------- */

uint32_t demora_ciclos = 300000;     /* con valor inicial -> sección .data */
uint32_t contador_parpadeos;         /* sin valor inicial -> sección .bss  */

/* ---------- Funciones ---------- */

static void demora(uint32_t ciclos)
{
    volatile uint32_t i;

    for (i = 0; i < ciclos; i++) {
        /* espera activa */
    }
}

/* Nadie la llama: sirve para ver qué hace --gc-sections */
void funcion_no_usada(void)
{
```

```
    contador_parpadeos = 0;
}

int main(void)
{
    SIM_SCGC5 = SIM_SCGC5 | SIM_SCGC5_PORTE;   /* 1. reloj del puerto E      */
    PORTE_PCR29 = PORT_PCR_MUX_GPIO;            /* 2. PTE29 como GPIO          */
    GPIOE_PSOR = LED_ROJO;                      /* 3. LED apagado (activo bajo) */
    GPIOE_PDDR = GPIOE_PDDR | LED_ROJO;         /* 4. PTE29 como salida        */

    while (1) {
        GPIOE_PTOR = LED_ROJO;                  /* invierte el LED */
        contador_parpadeos = contador_parpadeos + 1;
        demora(demora_ciclos);
    }
}
```

- Los `#define` de registros convierten una dirección en una variable: `GPIOE_PTOR = LED_ROJO` escribe en la dirección `0x400FF10C`. Es lo que hace un SDK, sin capas intermedias.
- `demora_ciclos` tiene **valor inicial**: va a la sección `.data`. `contador_parpadeos` no tiene valor inicial: va a `.bss`. La diferencia importa en el startup y en el mapa de memoria.
- `funcion_no_usada()` no la llama nadie. Sirve para ver cómo el linker la descarta con la opción `--gc-sections`.

3.2. El startup: lo que pasa antes de main()

Al salir de reset, el procesador no ejecuta `main()`: lee la **tabla de vectores** en la dirección 0 y salta a `Reset_Handler`. El archivo `startup_mkl46z4.c`, escrito en C para que se lea, contiene:

1. **La tabla de vectores** (sección `.isr_vector`, en `0x00000000`): 48 entradas. La primera es el valor inicial del stack pointer (`_estack`); la segunda, la dirección de `Reset_Handler`; luego los handlers de excepciones e interrupciones.
2. **El* flash configuration field* **(sección `.flash_config`, en `0x400`), propio de los Kinetis. El byte FSEC = `0xFE` deja el chip** sin proteger**. Un valor equivocado puede bloquearlo.
3. **`Reset_Handler`**: apaga el watchdog COP (que en el KL46 viene habilitado y resetearía el micro cada segundo), copia los valores iniciales de `.data` desde la FLASH a la RAM, pone `.bss` en cero y recién entonces llama a `main()`.

> **Un caso real de optimización:** los punteros de los lazos de `Reset_Handler` están declarados `volatile`. Sin eso, con `-Os` el compilador reconoce que los lazos copian y borran memoria y los **reemplaza por llamadas a `memcpy()` y `memset()`**, que no existen porque el programa se linkea sin biblioteca estándar (`-nostdlib`): el link falla con `undefined reference to memcpy`. Es un ejemplo de que el optimizador cambia el código, y de por qué el startup se escribe con cuidado.

3.3. El linker script: el archivo que describe el chip

```
MEMORY
{
  VECTORES (rx)  : ORIGIN = 0x00000000, LENGTH = 0x400
  FLASHCFG (rx)  : ORIGIN = 0x00000400, LENGTH = 0x10
  FLASH    (rx)  : ORIGIN = 0x00000410, LENGTH = 256K - 0x410
  RAM      (rwx) : ORIGIN = 0x1FFFE000, LENGTH = 32K
}
...
_sidata = LOADADDR(.data);
.data : { _sdata = .;  *(.data*)  _edata = .; } > RAM AT> FLASH
```

- **`MEMORY`** declara qué memorias tiene el chip, dónde empiezan y cuánto miden.
- **`SECTIONS`** indica en qué memoria va cada sección: el código (`.text`) en FLASH; las variables (`.data`, `.bss`) en RAM.

- **`> RAM AT> FLASH`**: `.data` *vive* en RAM pero se *guarda* en FLASH (al apagar, la RAM se borra). El startup la copia al arrancar.
- **Los símbolos** (`_sdata`, `_edata`, `_sbss`, `_ebss`, `_sidata`, `_estack`) los define el linker script y los usa el startup para saber qué copiar y qué borrar.

4. Preparación del entorno en Windows

4.1. Herramientas necesarias

Hacen falta cuatro programas. No hace falta instalar nada nuevo si ya están el **MCUXpresso IDE** y **ESP-IDF** (instalado con el EIM):

Herramienta	Dónde está (las versiones pueden variar)
`arm-none-eabi-gcc`	MCUXpresso: `C:\nxp\MCUXpressoIDE_25.6.136\ide\tools\bin`
`make`	MCUXpresso: `C:\nxp\MCUXpressoIDE_25.6.136\ide\buildtools\bin`
`cmake`	ESP-IDF: `C:\Espressif\tools\cmake\4.0.3\bin`
`ninja`	ESP-IDF: `C:\Espressif\tools\ninja\1.12.1`

Para encontrar las rutas en otra PC, en una ventana de cmd:

```
where /r C:\nxp arm-none-eabi-gcc.exe
where /r C:\nxp make.exe
where /r C:\Espressif cmake.exe
where /r C:\Espressif ninja.exe
```

`where /r carpeta archivo` busca el archivo dentro de esa carpeta y sus subcarpetas. Si el MCUXpresso devuelve dos rutas para el compilador, usar la que termina en `ide\tools\bin`. Si alguna herramienta no está, se puede instalar con `winget install Kitware.CMake`, `winget install Ninja-build.Ninja` o `winget install ezwinports.make`.

4.2. Descomprimir el proyecto

- Usar una ruta **sin espacios ni acentos**, por ejemplo `C:\sea2\`.
- Al descomprimir un `.zip`, Windows suele crear una carpeta dentro de otra con el mismo nombre (`blinky_kl46z\blinky_kl46z`). La carpeta del proyecto es **la de adentro**: la que contiene el `Makefile`.

4.3. Abrir la consola en la carpeta del proyecto

Se usa el **Símbolo del sistema** (`cmd`) de Windows, **no PowerShell**: los comandos de la práctica 1 usan la sintaxis de variables de cmd (`set` y `%CPU%`).

- **Forma rápida:** abrir la carpeta del proyecto en el Explorador, hacer clic en la barra de direcciones, escribir `cmd` y presionar Enter. Se abre la consola ya ubicada en esa carpeta.
- **Forma manual:** tecla Windows, escribir `cmd`, abrir *Símbolo del sistema* y moverse con `cd C:\sea2\blinky_kl46z`.
- **Para verificar** dónde se está: `dir` lista los archivos. Tiene que aparecer el `Makefile`.

4.4. Configurar el PATH

Cuando se escribe un comando como `make`, Windows lo busca en una lista de carpetas llamada **PATH**. Si la carpeta donde está el programa no figura en la lista, aparece el mensaje *"no se reconoce como un comando interno o externo"*. Las carpetas del MCUXpresso y de ESP-IDF no están en el PATH por defecto, así que hay que agregarlas.

El proyecto trae el archivo **`entorno.bat`**, que lo hace. Primero se editan en él las tres rutas (con el Bloc de notas) y después se ejecuta desde la consola:

```
@echo off
rem ================================================================
rem  Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
rem  Configura el PATH de ESTA ventana de cmd para compilar el blinky.
rem
rem  Uso: abrir cmd en la carpeta del proyecto y escribir   entorno
rem
rem  Ajustar las tres rutas a las de cada PC. Para encontrarlas:
rem    where /r C:\nxp arm-none-eabi-gcc.exe
rem    where /r C:\Espressif cmake.exe
rem    where /r C:\Espressif ninja.exe
rem ================================================================

set MCUX=C:\nxp\MCUXpressoIDE_25.6.136\ide
set CMAKE_DIR=C:\Espressif\tools\cmake\4.0.3\bin
set NINJA_DIR=C:\Espressif\tools\ninja\1.12.1

set PATH=%MCUX%\tools\bin;%MCUX%\buildtools\bin;%CMAKE_DIR%;%NINJA_DIR%;%PATH%

echo.
echo Herramientas encontradas:
where arm-none-eabi-gcc
where make
where cmake
where ninja
echo.
echo Si alguna no aparece, corregir su ruta en entorno.bat
```

```
entorno
```

> **Importante:** el PATH configurado así vale **solo para esa ventana de cmd**. Al cerrarla y abrir otra, hay que volver a ejecutar `entorno`. Para dejarlo fijo se pueden agregar las carpetas en *Configuración > Sistema > Información > Configuración avanzada del sistema > Variables de entorno*.

Verificación, de a un comando por vez:

```
arm-none-eabi-gcc --version      →  arm-none-eabi-gcc (Arm GNU Toolchain 14.2.Rel1 ...) 14.2.1
make --version                   →  GNU Make 4.4.1
cmake --version                  →  cmake version 4.0.3
ninja --version                  →  1.12.1
```

5. Práctica 1: el build a mano

En esta práctica **nosotros hacemos de build system**: ejecutamos cada etapa con su propio comando y elegimos el orden, los archivos y las opciones. Es exactamente lo que, en las prácticas siguientes, harán `make` y `cmake` por nosotros.

```
mkdir tmp
set CPU=-mcpu=cortex-m0plus -mthumb
arm-none-eabi-gcc %CPU% -E src/main.c -o tmp/main.i
arm-none-eabi-gcc %CPU% -Os -S src/main.c -o tmp/main.s
arm-none-eabi-gcc %CPU% -Os -c src/main.c -o tmp/main.o
arm-none-eabi-gcc %CPU% -Os -c startup/startup_mkl46z4.c -o tmp/startup.o
arm-none-eabi-gcc %CPU% -T linker/mkl46z256.ld -nostdlib tmp/main.o tmp/startup.o -o tmp/blinky.elf
arm-none-eabi-objcopy -O binary tmp/blinky.elf tmp/blinky.bin
```

Si un comando de `gcc` no muestra nada, es buena señal: solo escribe cuando hay errores o advertencias. Al terminar, `dir tmp` debe mostrar `main.i`, `main.s`, `main.o`, `startup.o`, `blinky.elf` y `blinky.bin`.

5.1. Qué hace cada comando

#	Comando	Qué hace (y qué reemplazará el build system)
1	`mkdir tmp`	Crea una carpeta para los archivos generados. En el build automático lo hace `make` (carpeta `build`).
2	`set CPU=...`	Comando de **cmd**, no de gcc: guarda las opciones del procesador en una variable para no repetirlas. `%CPU%` se reemplaza por su contenido. En el `Makefile` es la variable `CPU`.
3	`gcc -E`	**Etapa 1, preprocesar.** Genera `main.i`.
4	`gcc -Os -S`	**Etapa 2, compilar** (con optimización de tamaño). Genera `main.s`.
5	`gcc -Os -c src/main.c`	**Etapas 1 a 3** juntas: genera el objeto `main.o`.
6	`gcc -Os -c startup/...`	Lo mismo para el startup: `startup.o`. Hay un `.o` por cada `.c`.
7	`gcc -T ... -nostdlib ... -o blinky.elf`	**Etapa 4, linkear:** une los dos `.o` usando el linker script y genera el `.elf`.
8	`objcopy -O binary`	**Etapa 5, convertir:** extrae la imagen que se graba en la FLASH.

5.2. Qué significa cada opción

Opción	Significado
`-mcpu=cortex-m0plus`	Procesador destino: el núcleo del KL46Z.
`-mthumb`	Usar el juego de instrucciones Thumb (el único que ejecuta el Cortex-M0+).
`-E`, `-S`, `-c`	Detener el proceso después de preprocesar, compilar o ensamblar, respectivamente. Sin ninguna de ellas, gcc llega hasta el linkeo.
`-Os`	Optimizar el tamaño (ver 2.6).
`-o archivo`	Nombre del archivo de salida.
`-T archivo.ld`	Linker script a usar.
`-nostdlib`	No agregar la biblioteca estándar ni su startup: usamos el nuestro.
`-O binary`	(de `objcopy`) Formato de salida: binario crudo.

El nombre `arm-none-eabi-gcc` también tiene significado: **arm** es la arquitectura destino, **none** indica que no hay sistema operativo y **eabi** es la convención binaria para embebidos. Es un compilador **cruzado**: corre en la PC, pero genera código para otro procesador.

5.3. Qué observar en cada archivo

main.i: el código que ve el compilador

```
notepad tmp\main.i
```

Al final del archivo está `main()`, pero sin macros: cada registro se convirtió en su dirección.

```
while (1) {
    (*(volatile uint32_t *) 0x400FF10Cu) = (1u << 29);
    contador_parpadeos = contador_parpadeos + 1;
    demora(demora_ciclos);
}
```

main.s: el assembler

```
notepad tmp\main.s
```

Buscando `main:` aparecen las instrucciones Thumb del Cortex-M0+ (`movs`, `ldr`, `str`, `orrs`...). Es lo que se ve en la vista de desensamblado del depurador.

main.o: todavía sin direcciones

```
arm-none-eabi-nm tmp/main.o

00000000 B contador_parpadeos
00000000 D demora_ciclos
00000000 T funcion_no_usada
00000000 T main
```

`nm` lista los **símbolos** (funciones y variables) de un archivo. La letra indica la sección: **T** = código (`.text`), **D** = datos inicializados (`.data`), **B** = datos sin inicializar (`.bss`). Todos están en `00000000`: el `.o` no sabe todavía dónde va a quedar cada cosa.

blinky.elf: ya con direcciones

```
arm-none-eabi-nm -n tmp/blinky.elf

00000000 R tabla_vectores
00000400 R configuracion_flash
00000410 T funcion_no_usada
0000041c T main
00000482 T Reset_Handler
1fffe000 D demora_ciclos
1fffe004 B contador_parpadeos
20006000 R _estack
```

(Se muestran las líneas principales; `-n` ordena por dirección.) Ahora cada símbolo tiene su dirección, y coincide con el linker script: la tabla de vectores en 0, la configuración de la flash en `0x400`, el código desde `0x410` en FLASH, y las variables desde `0x1FFFE000` en RAM. **Las direcciones las puso el linker.** Notar que `funcion_no_usada` sigue en el binario: en este comando no se usó `--gc-sections` (ver práctica 3).

5.4. Experimento: linkear sin -nostdlib

```
arm-none-eabi-gcc %CPU% -T linker/mkl46z256.ld tmp/main.o tmp/startup.o -o tmp/error.elf
```

El link falla con varios errores, entre ellos:

```
undefined reference to `__bss_start__'
undefined reference to `__bss_end__'
```

Sin `-nostdlib`, gcc agrega por su cuenta el startup de la biblioteca estándar (`crt0`, de newlib), que espera símbolos que nuestro linker script no define. Conclusión: **el código que corre antes de `main()` no es mágico**; es un archivo más, escrito por alguien, que depende del linker script.

6. Práctica 2: explorar el binario

Para esta práctica se compila con `make` (práctica 3), que además genera el archivo `.map`. Si todavía no se usó, alcanza con ejecutar `make` una vez.

6.1. Cuánta memoria ocupa

```
make size

   text    data     bss     dec     hex filename
    380       4    1028    1412     584 build/blinky.elf
```

- **text**: código y constantes, más la tabla de vectores y la configuración de la flash (van a FLASH).
- **data**: variables con valor inicial (ocupan RAM, y su valor inicial también ocupa FLASH).
- **bss**: variables sin valor inicial, más el stack de 1 KB reservado (solo RAM).
- **FLASH usada** = text + data = 384 bytes. **RAM usada** = data + bss = 1032 bytes.

6.2. Dónde quedó cada sección

```
arm-none-eabi-objdump -h build/blinky.elf

Idx Name          Size      VMA       LMA
  0 .isr_vector   000000c0  00000000  00000000
  1 .flash_config 00000010  00000400  00000400
  2 .text         000000ac  00000410  00000410
  3 .data         00000004  1fffe000  000004bc
  4 .bss          00000004  1fffe004  000004c0
  5 .stack        00000400  1fffe008  000004c0
```

VMA es la dirección donde la sección *vive* durante la ejecución; **LMA**, donde se *carga* (se graba). Para casi todas coinciden, salvo **`.data`**: vive en RAM (`0x1FFFE000`) pero se graba en FLASH (`0x4BC`), y el startup la copia al arrancar. Es el `> RAM AT> FLASH` del linker script.

El archivo `build\blinky.map` tiene el mismo detalle, archivo por archivo. Buscando *Discarded input sections* aparece `.text.funcion_no_usada`: el linker la descartó gracias a `--gc-sections`.

6.3. Experimento: la RAM no alcanza

En `linker\mkl46z256.ld`, cambiar `LENGTH = 32K` de la RAM por `LENGTH = 1K`, guardar y ejecutar `make`:

```
region `RAM' overflowed by 8 bytes
```

Las variables (8 bytes) más el stack (1024 bytes) no entran en 1 KB. **El linker verifica en tiempo de compilación que todo entre en la memoria.** Es el mismo error que aparece cuando un proyecto con FreeRTOS reserva un heap demasiado grande. Volver a `32K` al terminar.

7. Práctica 3: automatizar con Make

Escribir los comandos a mano no escala: con muchos archivos es engorroso, y al modificar uno no se sabe qué hay que recompilar. **Make** resuelve ambas cosas. Lee el archivo `Makefile`, donde está escrito qué hay que generar y con qué comandos, y ejecuta solo los necesarios.

7.1. El Makefile

```
# Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
# Makefile del blinky bare-metal para la FRDM-KL46Z
#
# Uso:
#   make            compila en modo Release (-Os)
#   make DEBUG=1    compila en modo Debug (-O0 -g3)   ¡hacer "make clean" antes!
#   make size       muestra el uso de FLASH y RAM
#   make clean      borra la carpeta build
#
# Para crear y borrar carpetas se usa "cmake -E", que funciona igual en
# Windows (cmd o PowerShell) y en cualquier otro sistema.

# ---------- Toolchain ----------
PREFIJO  = arm-none-eabi-
CC       = $(PREFIJO)gcc
OBJCOPY  = $(PREFIJO)objcopy
SIZE     = $(PREFIJO)size

# ---------- Archivos ----------
BUILD    = build
FUENTES  = src/main.c startup/startup_mkl46z4.c
LDSCRIPT = linker/mkl46z256.ld
OBJETOS  = $(addprefix $(BUILD)/,$(FUENTES:.c=.o))

# ---------- Flags ----------
CPU      = -mcpu=cortex-m0plus -mthumb

ifeq ($(DEBUG),1)
OPT      = -O0 -g3
else
OPT      = -Os -g
endif

CFLAGS   = $(CPU) $(OPT) -Wall -ffunction-sections -fdata-sections -MMD -MP
LDFLAGS  = $(CPU) -T $(LDSCRIPT) -nostdlib -Wl,--gc-sections -Wl,-Map=$(BUILD)/blinky.map

# ---------- Reglas ----------
all: $(BUILD)/blinky.bin

$(BUILD)/blinky.bin: $(BUILD)/blinky.elf
    $(OBJCOPY) -O binary $< $@

$(BUILD)/blinky.elf: $(OBJETOS) $(LDSCRIPT)
    $(CC) $(LDFLAGS) $(OBJETOS) -o $@

$(BUILD)/%.o: %.c Makefile
    @cmake -E make_directory $(dir $@)
    $(CC) $(CFLAGS) -c $< -o $@

size: $(BUILD)/blinky.elf
    $(SIZE) $<

clean:
    cmake -E rm -rf $(BUILD)

.PHONY: all size clean

# Dependencias de headers generadas por gcc (-MMD -MP)
-include $(OBJETOS:.o=.d)
```

Un `Makefile` es una lista de **reglas** con esta forma:

```
objetivo: prerequisitos
<TAB>receta (comandos para generar el objetivo)
```

Si algún prerequisito es **más nuevo** que el objetivo (o el objetivo no existe), Make ejecuta la receta. Si no, no hace nada. La receta va indentada con **TAB**, no con espacios.

Elemento	Qué significa
`CC`, `CFLAGS`, `LDFLAGS`...	Variables: separan la configuración (compilador y opciones) de las reglas. `$(CC)` se reemplaza por su valor.
`ifeq ($(DEBUG),1)`	Elige las opciones de optimización: Release (`-Os -g`) o Debug (`-O0 -g3`) si se ejecuta `make DEBUG=1`.
`$(BUILD)/blinky.elf: $(OBJETOS) $(LDSCRIPT)`	El `.elf` depende de los `.o` **y del linker script**: si cambia el `.ld`, se vuelve a linkear.
`$(BUILD)/%.o: %.c Makefile`	**Regla patrón**: una sola regla sirve para todos los `.c`. Depende también del `Makefile`.
`$@` y `$<`	Variables automáticas: el objetivo de la regla y su primer prerequisito.
`-MMD -MP` e `-include`	gcc genera archivos `.d` con los headers que usa cada `.c`; así, si cambia un `.h`, se recompila lo que lo incluye.
`cmake -E make_directory`, `cmake -E rm -rf`	Crean y borran carpetas igual en cmd, PowerShell o Linux: hacen que el mismo `Makefile` funcione en Windows.
`.PHONY`	Objetivos que no son archivos (`all`, `size`, `clean`).

> **Lo que reemplaza Make:** las líneas que `make` imprime al compilar son **los mismos comandos de gcc de la práctica 1**, con algunas opciones más. Make no compila: escribe los comandos por nosotros y decide cuáles hacen falta.

7.2. Paso a paso

1. Compilar todo

```
make clean
make


arm-none-eabi-gcc -mcpu=cortex-m0plus -mthumb -Os -g -Wall -ffunction-sections -fdata-sections -MMD
-MP
    -c src/main.c -o build/src/main.o
arm-none-eabi-gcc ... -c startup/startup_mkl46z4.c -o build/startup/startup_mkl46z4.o
arm-none-eabi-gcc -mcpu=cortex-m0plus -mthumb -T linker/mkl46z256.ld -nostdlib -Wl,--gc-sections
    -Wl,-Map=build/blinky.map build/src/main.o build/startup/startup_mkl46z4.o -o build/blinky.elf
arm-none-eabi-objcopy -O binary build/blinky.elf build/blinky.bin
```

Son los pasos de la práctica 1. Las opciones nuevas: `-Wall` activa las advertencias; `-ffunction-sections -fdata-sections` ponen cada función y variable en su propia sección, y `-Wl,--gc-sections` le indica al linker que descarte las que nadie usa; `-Wl,-Map=...` genera el `.map`. (`-Wl,` pasa la opción al linker.)

2. Compilar otra vez

```
make

make: Nothing to be done for 'all'.
```

Todo está al día: no hace nada. Este es el **build incremental**.

3. Modificar main.c y compilar

```
copy /b src\main.c +,,
make
```

`copy /b archivo +,,` es un truco del cmd que actualiza la **fecha** del archivo sin cambiar su contenido (el equivalente de `touch` en Linux). Make recompila **solo `main.c`**, vuelve a linkear y genera el `.bin`; el startup no se toca. Lo mismo pasa si se edita y guarda `main.c`.

4. La trampa: cambiar a Debug sin limpiar

```
make DEBUG=1

make: Nothing to be done for 'all'.
```

Se pidieron otras opciones de optimización (`-O0 -g3`), pero Make **no recompila nada**: el `.elf` sigue siendo el de Release. Make decide qué recompilar comparando **fechas**, no **opciones**; como los `.c` no cambiaron, todo le parece al día.

> **Por qué importa:** quien cree estar depurando en Debug está, en realidad, depurando el binario optimizado: los breakpoints saltan y las variables aparecen como *optimized out*. Con Make, al cambiar de modo **hay que hacer `make clean`**. CMake lo resuelve de otra forma (práctica 4).

5. Debug, ahora sí

```
make clean
make DEBUG=1
make size

   text    data     bss     dec     hex filename
    472       4    1028    1504     5e0 build/blinky.elf
```

Sin optimización el código pasa de 380 a 472 bytes.

6. Volver a Release

```
make clean
make
```

8. Práctica 4: CMake y Ninja

CMake no compila: genera el build. A partir de una descripción del proyecto (`CMakeLists.txt`) escribe las instrucciones para otro programa —en este caso **Ninja**— que es el que ejecuta los comandos de gcc. Por eso con CMake hay dos pasos: **configurar** y **compilar**.

```
CMakeLists.txt + toolchain ──► cmake (configura) ──► build.ninja ──► ninja (compila) ──► blinky.elf
/ .bin
```

8.1. El toolchain file

```
# Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
# Toolchain file: le dice a CMake que compile para ARM y no para la PC.

set(CMAKE_SYSTEM_NAME Generic)          # sin sistema operativo
set(CMAKE_SYSTEM_PROCESSOR arm)

set(CMAKE_C_COMPILER   arm-none-eabi-gcc)
set(CMAKE_ASM_COMPILER arm-none-eabi-gcc)
set(CMAKE_OBJCOPY      arm-none-eabi-objcopy)
set(CMAKE_SIZE         arm-none-eabi-size)

# CMake prueba el compilador armando un ejecutable. Sin linker script fallaría:
# así prueba armando una biblioteca estática.
set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)
```

Le dice a CMake que compile **para ARM y no para la PC**: sin sistema operativo (`Generic`) y con `arm-none-eabi-gcc`. Está separado del proyecto, de modo que el mismo `CMakeLists.txt` serviría con otra toolchain. Es el equivalente de la variable `PREFIJO = arm-none-eabi-` del `Makefile`.

8.2. El CMakeLists.txt

```
# Sistemas Embebidos Avanzados II - DSI - EIE - FCEIA - UNR
# CMakeLists del blinky bare-metal para la FRDM-KL46Z
#
# Configurar (una vez por carpeta de build):
#   cmake -B build-release -G Ninja -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake -
DCMAKE_BUILD_TYPE=Release
#   cmake -B build-debug   -G Ninja -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake -
DCMAKE_BUILD_TYPE=Debug
# Compilar (cada vez):
#   cmake --build build-release

cmake_minimum_required(VERSION 3.16)
project(blinky_kl46z C)

set(LDSCRIPT ${CMAKE_SOURCE_DIR}/linker/mkl46z256.ld)

# Opciones de cada tipo de build
set(CMAKE_C_FLAGS_DEBUG   "-O0 -g3")
set(CMAKE_C_FLAGS_RELEASE "-Os -g")

add_executable(blinky.elf
    src/main.c
    startup/startup_mkl46z4.c
)

target_compile_options(blinky.elf PRIVATE
    -mcpu=cortex-m0plus -mthumb -Wall
    -ffunction-sections -fdata-sections
)

target_link_options(blinky.elf PRIVATE
    -mcpu=cortex-m0plus -mthumb
```

```
    -T ${LDSCRIPT}
    -nostdlib
    -Wl,--gc-sections
    -Wl,-Map=${CMAKE_BINARY_DIR}/blinky.map
)

# Si cambia el linker script, volver a linkear
set_target_properties(blinky.elf PROPERTIES LINK_DEPENDS ${LDSCRIPT})

# Después de linkear: generar el .bin y mostrar el uso de memoria
add_custom_command(TARGET blinky.elf POST_BUILD
    COMMAND ${CMAKE_OBJCOPY} -O binary blinky.elf blinky.bin
    COMMAND ${CMAKE_SIZE} blinky.elf
    WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
)
```

A diferencia del `Makefile`, no se escriben reglas ni comandos: se **describe** el resultado (un ejecutable `blinky.elf`, sus fuentes y sus opciones) y CMake se ocupa de las dependencias, las carpetas y el comando de link.

Línea	Qué hace
`add_executable(blinky.elf ...)`	El programa a generar y sus fuentes.
`target_compile_options(...)`	Opciones de compilación (las mismas que `CFLAGS`).
`target_link_options(...)`	Opciones de link: linker script, `-nostdlib`, `--gc-sections`, `.map`.
`CMAKE_C_FLAGS_DEBUG` / `_RELEASE`	Opciones de optimización de cada modo.
`LINK_DEPENDS`	Volver a linkear si cambia el linker script.
`add_custom_command(... POST_BUILD ...)`	Después de linkear: generar el `.bin` y mostrar el tamaño.

8.3. Paso a paso

1. Configurar: una vez por carpeta de build

```
cmake -B build-release -G Ninja -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake -
DCMAKE_BUILD_TYPE=Release
cmake -B build-debug   -G Ninja -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake -
DCMAKE_BUILD_TYPE=Debug
```

Opción	Significado
`-B build-release`	Carpeta donde se genera el build (se crea si no existe).
`-G Ninja`	Generar instrucciones para Ninja (podría ser `"Unix Makefiles"` para Make).
`-DCMAKE_TOOLCHAIN_FILE=...`	Toolchain a usar: el archivo de la sección 8.1.
`-DCMAKE_BUILD_TYPE=Release`	Modo de compilación: Release o Debug.

2. Compilar: cada vez que se modifica el código

```
cmake --build build-release

[1/3] Building C object CMakeFiles/blinky.elf.dir/src/main.c.obj
[2/3] Building C object CMakeFiles/blinky.elf.dir/startup/startup_mkl46z4.c.obj
[3/3] Linking C executable blinky.elf
   text    data     bss     dec     hex filename
    380       4    1028    1412     584 blinky.elf
```

```
cmake --build build-debug

    472       4    1028    1504     5e0 blinky.elf
```

`cmake --build` llama a `ninja` en esa carpeta. Ninja muestra un resumen de cada paso en lugar del comando completo (con `cmake --build build-release -v` se ven los comandos de gcc).

> **La trampa de Make, resuelta:** cada modo tiene **su propia carpeta**, con sus opciones y sus archivos. Release y Debug conviven sin pisarse y no hace falta limpiar al cambiar de uno a otro.

3. Experimento: olvidar el toolchain file

```
cmake -B build-error -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build-error
```

Sin toolchain file, CMake usa el compilador de la PC, que no conoce las opciones del Cortex-M0+: la compilación falla con un error como *unrecognized command-line option '-mthumb'* (si la PC no tiene ningún compilador de C instalado, ya falla al configurar). Borrar la carpeta `build-error` al terminar.

9. Grabar el programa en la placa

Con el MCUXpresso IDE, la forma habitual en la materia:

1. Abrir el MCUXpresso y seleccionar en el *Project Explorer* cualquier proyecto de la KL46Z (le indica al IDE el microcontrolador).
2. Hacer clic en **GUI Flash Tool** y elegir la sonda detectada.
3. En la pestaña **Program**, *File to program*: **Browse** y elegir `build\blinky.elf`. Con el `.elf` no hace falta indicar la dirección: viene adentro.
4. **Run** y, al terminar, presionar **RESET** en la placa. El LED rojo debe parpadear.

Arrastrando el archivo: algunas aplicaciones OpenSDA (MSD, mbed, DAPLink) hacen que la placa aparezca como una unidad USB al conectarla normalmente; en ese caso se graba copiando `build\blinky.bin` a esa unidad. Con una aplicación solo de depuración (P&E o J-Link) no aparece ninguna unidad y se usa el MCUXpresso.

> **Atención:** si la placa se conecta con el botón **RESET presionado**, aparece la unidad **BOOTLOADER**. Esa unidad sirve para actualizar el firmware del **depurador** (la aplicación OpenSDA), no el programa del KL46Z. **No copiar el blinky ahí.**

10. Del blinky al IDE y a los SDK

10.1. El botón Build del MCUXpresso

Compilando cualquier proyecto de la KL46Z en el MCUXpresso y abriendo su carpeta **`Debug`** aparece todo lo visto: los makefiles que generó el IDE (`makefile`, `sources.mk`, `objects.mk` y un `subdir.mk` por carpeta), los linker scripts que generó (archivos `.ld`), el `.map` y el `.axf`, que es un `.elf` con otra extensión. Al presionar Build, el IDE ejecuta `make` con esos archivos.

10.2. ESP-IDF y Zephyr

Los SDK actuales usan las mismas ideas a mayor escala. En ESP-IDF, el botón de VS Code ejecuta `idf.py`, que configura con **CMake** y compila con **Ninja**; el proyecto se organiza en componentes y se configura con **Kconfig**

(menuconfig). Zephyr usa CMake, Kconfig y devicetree, invocados con `west`. Lo que en esta práctica se hizo con cinco archivos, esos SDK lo hacen con miles, con las mismas herramientas.

11. Problemas frecuentes

Síntoma	Causa y solución
"arm-none-eabi-gcc" no se reconoce como un comando interno o externo (o lo mismo con `make`, `cmake`, `ninja`)	La carpeta de la herramienta no está en el PATH. Ejecutar `entorno` (sección 4.4) o buscar la ruta con `where /r`.
El PATH funcionaba y dejó de funcionar	Se abrió otra ventana de cmd: el PATH configurado con `set` vale solo para la ventana donde se ejecutó. Volver a ejecutar `entorno`.
`make: *** No targets specified and no makefile found. Stop.`	La consola no está en la carpeta del proyecto. Con `dir` verificar que aparezca el `Makefile`; revisar si quedó una carpeta dentro de otra al descomprimir.
Los comandos de la práctica 1 fallan con `%CPU%`	Se está usando PowerShell. Usar el Símbolo del sistema (cmd).
`make` falla con un error de `cmake`	El `Makefile` usa `cmake -E` para crear carpetas: `cmake` tiene que estar en el PATH.
`undefined reference to memcpy` / `memset`	Se sacó el `volatile` de los punteros de `Reset_Handler` (sección 3.2).
`undefined reference to __bss_start__`	Se linkeó sin `-nostdlib` (sección 5.4).
`region 'RAM' overflowed`	Las variables y el stack no entran en la RAM declarada en el linker script.
`make DEBUG=1` no recompila	Es la trampa de la sección 7.2: hacer `make clean` antes.
CMake falla con *unrecognized command-line option '-mthumb'*	Falta `-DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake` al configurar. Borrar la carpeta de build y configurar de nuevo.
Errores con rutas extrañas	El proyecto está en una ruta con espacios o acentos. Moverlo, por ejemplo, a `C:\sea2\`.

12. Referencia rápida de comandos

Comando	Herramienta	Para qué
`arm-none-eabi-gcc`	Toolchain	Preprocesar, compilar, ensamblar y linkear (según la opción)
`arm-none-eabi-objcopy -O binary`	Toolchain	Convertir el `.elf` en `.bin`
`arm-none-eabi-nm [-n]`	Toolchain	Listar símbolos [ordenados por dirección]
`arm-none-eabi-size`	Toolchain	Tamaño de text, data y bss
`arm-none-eabi-objdump -h` / `-d`	Toolchain	Tabla de secciones / desensamblado
`make`, `make clean`, `make size`, `make DEBUG=1`	Build system	Compilar, limpiar, ver tamaños, compilar en Debug
`cmake -B carpeta ...`	Build system	Configurar (generar las instrucciones)
`cmake --build carpeta`	Build system	Compilar (llama a ninja)
`set`, `%VAR%`, `cd`, `dir`, `mkdir`, `where /r`, `copy /b ... +,,`	cmd de Windows	Variables, carpetas, buscar archivos, actualizar la fecha de un archivo
`entorno`	cmd (archivo .bat del proyecto)	Configurar el PATH de la ventana

13. Ejercicios

1. Separar el manejo del LED en `led.c` y `led.h` y agregarlo al `Makefile` y al `CMakeLists.txt`. ¿Cómo detecta cada herramienta el archivo nuevo?
2. Hacer parpadear también el LED verde (PTD5): habilitar el reloj del puerto D (bit 12 de `SIM_SCGC5`) y usar los registros de `GPIOD` (base `0x400FF0C0`).
3. Compilar con `make` y con los comandos de la práctica 1, y comparar con `nm -n` dónde quedó `main` en cada caso. Explicar la diferencia.
4. Achicar la RAM a 1K en el linker script, explicar el error y calcular cuántos bytes hacen falta como mínimo.
5. Sacar el `volatile` de los punteros de `Reset_Handler` y compilar. ¿Por qué aparecen `memcpy` y `memset`?
6. En el desensamblado de `Reset_Handler` (`arm-none-eabi-objdump -d build/blinky.elf`), encontrar los lazos que copian `.data` y borran `.bss`.
7. En la carpeta `Debug` de un proyecto de FreeRTOS de la KL46Z, buscar en el `.map` cuánto ocupa el heap de FreeRTOS (`ucHeap`).

14. Referencias

- GNU. *GCC — Using the GNU Compiler Collection* (opciones de compilación y optimización). gcc.gnu.org/onlinedocs
- GNU. *GNU Make Manual*. gnu.org/software/make/manual
- GNU. *The GNU Linker (ld)* — linker scripts. sourceware.org/binutils/docs/ld
- Kitware. *CMake Documentation* — cmake-toolchains. cmake.org/cmake/help/latest
- Ninja. *The Ninja build system*. ninja-build.org/manual.html
- NXP. *KL46 Sub-Family Reference Manual* (KL46P121M48SF4RM).
- NXP. *FRDM-KL46Z User's Guide*.
- Arm. *Arm GNU Toolchain*. developer.arm.com